

Priority Queues at Work

Programming and Data Structures — Week 8, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement a priority queue as a thin, disciplined interface layered over the heap built in Lecture 2, mirroring exactly how Week 3 built stacks and queues as disciplined interfaces over Week 2's dynamic array; use Python's built-in `heapq` module correctly, including the common trick needed to simulate a max-heap; trace a task scheduler processing tasks strictly by priority, completely, on a concrete example; and preview the core loop of Dijkstra's shortest-path algorithm, explaining precisely why a heap is the structurally correct choice for it, ahead of Week 12's full treatment.

Outline

Today covers the disciplined-interface pattern recalled directly from Week 3; building `PriorityQueue` on top of Lecture 2's heap; Python's `heapq` module, what it actually provides, and the standard trick for simulating a max-heap using it; a task scheduler application, traced completely; merging k sorted lists using a priority queue, connecting back to Week 6's merge sort; a preview of Dijkstra's algorithm's core loop; the MTech-track material on building a stable priority queue that breaks ties correctly; an in-class quiz; and a summary.

The disciplined-interface idea, recalled from Week 3

Week 3 established that `Stack` and `Queue` were not new storage mechanisms in their own right, but restricted, disciplined interfaces layered over Week 2's dynamic array, exposing only `push/pop` or `enqueue/dequeue` rather than general array access. This lecture repeats that exact pattern one level up: `PriorityQueue` is a restricted interface over Lecture 2's heap, exposing only `insert` and `extract_min`, never direct array-index manipulation. The restriction itself is the value being added — hiding heap-internal arithmetic behind clearly named operations prevents a caller from accidentally violating the heap property directly.

PriorityQueue, built on Lecture 2's heap

```
class PriorityQueue:
    def __init__(self):
        self._heap = MinHeap()          # Lecture 2's structure, reused directly

    def push(self, priority, item):
        self._heap.insert((priority, item))  # tuples compare element-wise: priority first

    def pop(self):
        priority, item = self._heap.extract_min()
        return item

    def is_empty(self):
        return len(self._heap.data) == 0
```

Storing (priority, item) tuples is a deliberate design choice: Python compares tuples element by element, so comparing two such tuples compares their priorities first automatically, exactly matching the desired ordering without any custom comparison logic. `pop()` always returns the item associated with the smallest priority value — “smallest number means most urgent” is a convention adopted here, not an inherent property of priority queues; a max-priority convention is equally valid and equally common. Every operation's cost is inherited directly from Lecture 2's analysis, since `PriorityQueue` performs no work of its own beyond packing and unpacking tuples.

Python's `heapq`: what it actually provides

```
import heapq

data = [5, 3, 8, 1, 9]
heapq.heapify(data)          # in-place, O(n) - Lecture 2's MTech addition
heapq.heappush(data, 2)      # O(log n) - Lecture 2's heapify_up
smallest = heapq.heappop(data) # O(log n) - Lecture 2's heapify_down
```

Python's standard library `heapq` module operates directly on a plain list, with no separate heap class wrapping it at all — exactly the array-backed design this course built from scratch in Lecture 2. `heapq.heapify` is Lecture 2's $O(n)$ heapify algorithm; `heapq.heappush` is `heapify_up`; `heapq.heappop` is `heapify_down`. `heapq` provides only min-heap behavior — there is no built-in max-heap variant, which the next slide addresses directly.

The max-heap trick

```
import heapq

max_heap = []
for value in [5, 3, 8, 1, 9]:
    heapq.heappush(max_heap, -value)      # negate on the way IN

largest = -heapq.heappop(max_heap)       # negate again on the way OUT
```

Since `heapq` supports only min-heap behavior, simulating a max-heap uses a standard trick: negate every value before inserting it, which flips the ordering entirely — the originally largest value, once negated, becomes the most negative, and therefore the heap's minimum. Reading a value back out requires negating again to undo this transformation and recover the original value. This is a small, well-known technique worth recognizing immediately when encountered in code, since `heapq` itself provides no direct max-heap alternative.

Why tuples compare the way they do — a quick digression

It is worth pausing on exactly why storing `(priority, item)` tuples works so cleanly. Python compares tuples lexicographically, exactly like dictionary word ordering: `(3, "a")` is less than `(5, "b")` because `3 < 5` alone already decides the comparison, with the second elements never even examined. `(3, "a")` versus `(3, "b")` ties on the first element, so Python falls through to comparing the second elements, `"a" < "b"`. This is precisely why packing priority first into the tuple guarantees priority is always the decisive factor unless two priorities are exactly equal — the exact case this lecture's stable variant addresses later.

Application: a task scheduler, traced completely

```
scheduler = PriorityQueue()
scheduler.push(3, "send weekly report")
scheduler.push(1, "handle server outage")
scheduler.push(2, "review pull request")

while not scheduler.is_empty():
    print(scheduler.pop())
```

Output:

handle server outage	(priority 1 - most urgent)
review pull request	(priority 2)
send weekly report	(priority 3 - least urgent)

Tracing this task scheduler: three tasks are pushed with priorities 3, 1, and 2 respectively, in that arrival order — notice the arrival order does not match priority order at all. Popping repeatedly always returns the currently most urgent remaining task first: “handle server outage” (priority 1), then “review pull request” (priority 2), then “send weekly report” (priority 3) — correctly processing by urgency rather than arrival order, exactly the capability the hospital-triage analogy in Lecture 2 motivated.

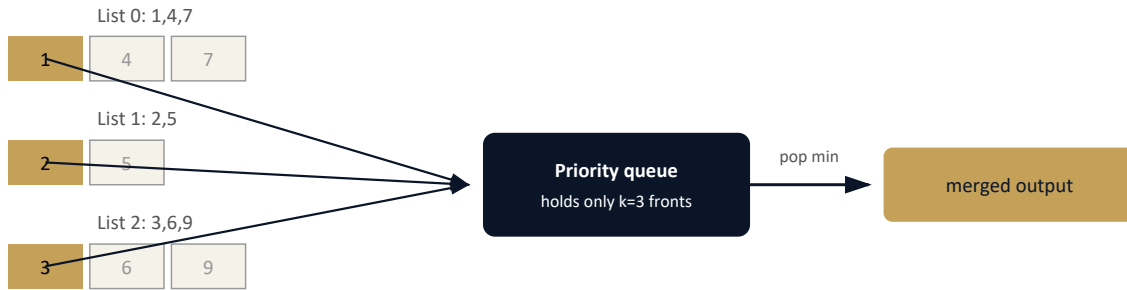
Application: merging k sorted lists, connecting to Week 6

```
def merge_k_sorted(lists):
    pq = PriorityQueue()
    for i, lst in enumerate(lists):
        if lst:
            pq.push(lst[0], (i, 0))          # (value, (list_index, element_index))

    result = []
    while not pq.is_empty():
        i, idx = pq.pop()                   # pop() returns the ITEM only, per this lecture's P
        value = lists[i][idx]               # look the value up directly - it was never lost,
        result.append(value)
        if idx + 1 < len(lists[i]):
            pq.push(lists[i][idx + 1], (i, idx + 1))
    return result
```

Week 6 Lecture 1’s merge function combined exactly two sorted lists in $O(m + n)$ time. Merging k sorted lists by naively applying that merge function repeatedly, pairwise, costs $O(nk)$ total for n total elements across all lists. Using a priority queue instead, holding only the current front element of each of the k lists at any moment, the next overall smallest element is found via a single $O(\log k)$ pop, and that list’s next element is pushed back in, also $O(\log k)$. Total cost: $O(n \log k)$ — genuinely better than the naive approach when k is large, a direct improvement earned by combining Week 6’s merge idea with this week’s heap.

The picture: k lists, one priority queue holding the fronts



Only the current front of each list ever sits in the queue — its size stays bounded by k , not n .

Only the current front element of each list ever sits in the priority queue at once — this is what keeps the queue's size bounded by k , the number of lists, rather than n , the total number of elements, giving the $O(\log k)$ per-step cost.

Tracing merge_k_sorted on a small example, completely

Tracing `merge_k_sorted` completely on three small lists: the priority queue never holds more than 3 elements at once — one front element per list — confirming the $O(\log k)$ per-step claim directly, since $k = 3$ here regardless of how many total elements exist across all lists. Each pop correctly identifies the smallest remaining front value across all lists, and immediately refills from whichever list that value came from, producing the fully sorted 8-element result `[1, 2, 3, 4, 5, 6, 7, 9]`.

Preview: Dijkstra's algorithm's core loop

```
# Preview only - full treatment and correctness proof in Week 12
def dijkstra_preview(graph, start):
    distances = {start: 0}
    pq = PriorityQueue()
    pq.push(0, start)
    while not pq.is_empty():
        current = pq.pop()
        for neighbor, weight in graph[current]:
            new_dist = distances[current] + weight
            if neighbor not in distances or new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
```

```
        pq.push(new_dist, neighbor)    # push the BETTER distance found
    return distances
```

Dijkstra’s shortest-path algorithm, covered formally in Week 12, repeatedly asks: of everywhere reachable so far, what is the closest place not yet finalized? This is exactly this lecture’s priority queue query — “give me the current minimum” — applied to accumulated graph distances instead of task urgency or list-merge values. This lecture previews only the structural reason a heap is the right tool for this loop; Week 12 supplies the actual graph representation and the correctness proof that this “always expand the currently closest” greedy strategy produces genuinely shortest paths.

MTech addition — a stable priority queue

```
class StablePriorityQueue:
    def __init__(self):
        self._heap = MinHeap()
        self._counter = 0          # tie-breaker, strictly increasing

    def push(self, priority, item):
        self._heap.insert((priority, self._counter, item))
        self._counter += 1

    def pop(self):
        priority, _, item = self._heap.extract_min()
        return item
```

A subtle bug in the plain `PriorityQueue` implementation: if two items share the same priority, Python’s tuple comparison falls through to comparing the `item` elements directly, which crashes if items are not themselves comparable — two dictionaries, for instance. Adding a strictly increasing counter as the tuple’s second element guarantees every tie is broken by insertion order before Python ever needs to compare the `items` themselves, since no two counters are ever equal. This is precisely Week 6 Lecture 1’s stability concept, reapplied here: equal-priority items are processed in the order they were originally pushed, and the underlying items never need to support comparison at all.

Real-world use: `asyncio` and OS process scheduling

Python’s own `asyncio` event loop uses a `heapq`-based priority queue internally to schedule delayed callbacks, ordering them by their scheduled wake-up time — precisely this lecture’s `PriorityQueue` pattern, running inside real, widely-used infrastructure. Real operating system process schedulers, a topic Week 14’s parallel and randomized algorithms lecture touches on, often use priority-queue-like

structures to prioritize processes by urgency rather than strict arrival order, for exactly the reasons the hospital-triage analogy in Lecture 2 introduced. This lecture's task scheduler example is not merely an illustrative analogy — it is a simplified version of a mechanism genuinely running inside real systems today.

Exercise

As an exercise, implement `merge_k_sorted` exactly as presented in this lecture, and confirm on several test cases that it produces identical output to calling Python's built-in `sorted()` on the flattened concatenation of all input lists. Separately, trace the max-heap trick by hand: push the values `[3, 1, 4, 1, 5]` using negation, and confirm that popping five times in a row returns them in descending order, `[5, 4, 3, 1, 1]`.

A second worked trace — the max-heap trick, verified

Verifying the max-heap trick completely on `[3, 1, 4, 1, 5]`: negating every value before insertion means the heap's actual minimum, `-5` (corresponding to the original maximum, `5`), is popped first, and negating the result on the way out correctly recovers `5`. Continuing, the sequence recovers `4`, then `3`, then the two `1`s in some order, producing `[5, 4, 3, 1, 1]` — genuinely descending order, extracted entirely using `heapq`'s min-heap-only machinery.

Quiz — trace the scheduler

Given tasks pushed in the order `(5, "A")`, `(2, "B")`, `(2, "C")`, `(8, "D")` into the plain, non-stable `PriorityQueue` from this lecture, determine the order in which they are popped, before checking the answer.

Quiz — answer

Items pop in ascending priority order: both priority-2 tasks first, then priority 5, then priority 8. Between "B" and "C," which share priority 2, the plain (non-stable) `PriorityQueue` implementation falls back to comparing the strings "B" and "C" directly as a tie-breaker, which happens to work correctly here since strings are comparable — but this is not a general guarantee, and would crash on non-comparable items, exactly the motivation for this lecture's MTech stable variant. The correct final order is "B", then "C" (or vice versa, depending on string comparison), then "A", then "D".

Cost summary for this lecture's applications

Consolidating every cost established in this lecture: individual priority-queue operations cost $O(\log n)$, inherited directly from Lecture 2; building a heap from an existing array costs $O(n)$, using Lecture 2's `heapify`; and this lecture's k -way merge costs $O(n \log k)$ total, a genuine improvement over the $O(nk)$ a naive repeated pairwise application of Week 6 Lecture 1's merge would cost when k is large relative to $\log k$ — the exact condition under which this lecture's approach earns its keep.

Summary

`PriorityQueue` is a disciplined interface layered directly over Lecture 2's heap, exactly mirroring the pattern Week 3 used to build stacks and queues over Week 2's dynamic array. Python's `heapq` module operates in place on a plain list and provides only min-heap behavior, requiring the standard negation trick to simulate a max-heap. Task scheduling and merging k sorted lists are genuine applications, the latter a direct, quantifiable improvement over naively repeating Week 6's pairwise merge. Dijkstra's shortest-path algorithm's core loop is structurally identical to this lecture's repeated "give me the current minimum" priority queue query, with Week 12 supplying the full graph context and correctness proof. Next lecture closes this unit by choosing between hash tables, heaps, and Week 7's balanced trees for real, concretely described workloads.